

MODULE 23 · PART 8 - PRACTICE

Diagnostics

Eight wrong results, handed over cold, with no clue which module they came from.

PART 8 of 8 MODULE 23 of 26 RUNS about 25 min

Why diagnosis is its own skill

- ◆ **Writing a correct query and finding a wrong one are different jobs**, and only the first is ever taught.
- ◆ **Most of a data engineer's week is the second**, because most queries already exist and somebody is querying a number.
- ◆ **And the search space is the difficulty**, not the techniques. Every technique here you already know.
- ◆ **So the skill is narrowing**, fast, with observations that cost nothing.

THE TAKEAWAY

Twenty-two modules taught you what can go wrong. This one is about finding out which of those twenty-two did, from a symptom and nothing else.

In a module about frames, **the answer is about frames**. That is not a property of real work.

Which changes the method entirely

- ◆ **You cannot check twenty-two techniques**, one at a time, against every defect. That is a day per defect.
- ◆ **So work from the symptom rather than the technique**, because the symptom narrows the search far faster.
- ◆ **A handful of symptoms cover most defects**, and each points at two or three causes rather than twenty-two.
- ◆ **This module is that mapping**, built backwards from every scenario in the course.

THE TAKEAWAY

The course taught technique to symptom. Diagnosis needs symptom to technique, and that is a different index over the same material.

Four questions, asked in this order

QUESTION	WHAT THE ANSWER RULES OUT
1. Too many rows, too few, or wrong values?	Two thirds of everything, in one question.
2. Is the discrepancy an exact multiple?	An exact integer means counting per level. Untidy means data.
3. Does it reconcile against itself?	If yes, the arithmetic is fine and the population is wrong.
4. Is the error concentrated or spread?	At the start means history. At edges means partitions.

THE TAKEAWAY

Four questions, none of which requires reading the SQL. They will usually get you to two or three candidate causes before you open the query at all.

Why the order matters

- ◆ **Question one is free**, needs one row count, and eliminates more than any other single observation.
- ◆ **Question two is nearly free**, and an exact multiple is such a strong signal it usually ends the investigation.
- ◆ **Question three is the one people start with**, and a result reconciling against itself is the least informative outcome.
- ◆ **Question four needs a breakdown**, so it is last, and it is what you use when the first three did not settle it.

THE TAKEAWAY

Starting with 'does it reconcile' is the common instinct, and a yes means only that the query is consistent with itself, which every wrong query also is.

Eight diagnostics. **Pause on each one properly**, because the pause is the whole exercise.

The first query to run

```
1 -- before reading any SQL, before forming any theory
2
3 SELECT
4     (SELECT COUNT(*) FROM the_result) AS rows_out,
5     (SELECT COUNT(*) FROM the_source) AS rows_in,
6     (SELECT ROUND(SUM(measure),2) FROM the_result) AS total_out,
7     (SELECT ROUND(SUM(measure),2) FROM the_source) AS total_in;
8
9 -- four numbers, and two ratios you can compute in your head
```

5 Rows first, because it answers question one and question one eliminates two thirds of everything.

THE TAKEAWAY

Four numbers. If rows_out exceeds rows_in you have a fan-out; if it is lower you have a loss; if they match the values are the problem.

What the row count tells you

- ◆ **More rows out than in: a fan-out.** A join, an interval that overlaps, an unnest, an expansion.
- ◆ **Fewer rows out: a loss.** An inner join, a filter, an empty container, a deduplication.
- ◆ **Equal rows: the values.** Frames, denominators, windows, ordering. A much harder class.
- ◆ **And both at once is common**, so compare against a known count rather than only against the input.

THE TAKEAWAY

The last point is module fifteen's lesson: a closed-interval join both duplicated 40,930 rows and lost 2.09 million, and the total looked plausible because they partly cancelled.

The exact-multiple test

```
1 observed / expected = 3.000000 -> per-level counting
2 observed / expected = 2.000000 -> a two-way fan-out, or a double load
3 observed / expected = 1.000000... -> the rows are fine
4
5 observed / expected = 1.6183 -> data. An interrupted rerun,
6                               a partial load, real duplicates
7
8 -- compute it to six decimal places. The tidiness is the signal.
```

5 1.6183 is not a clean fraction of anything. Untidy ratios are data problems; tidy ones are structural.

THE TAKEAWAY

A tidy ratio means something in the query counts a thing more than once, deterministically. An untidy one means the data has something in it you do not know about.

Ask what the defect **correlates with** before asking what the query does.

A regional summary published for reporting shows a company total of exactly 902,431,118. Finance's figure is 300,810,373. Spot checks confirm every individual row in the table is correct, including the totals rows.

The dashboard sums the `net_sales` column with no filter applied.

What is the cause?

- ☐ A The fact table contains duplicate rows, which the summary is faithfully reproducing.
- ☐ B The result contains subtotals and a grand total, so every sale is counted once per grouping level.
- ☐ C The load ran three times, and the summary was rebuilt from the tripled source.
- ☐ D A join has fanned out against a dimension with three rows per key.

◆ PAUSE AND DECIDE

What is the cause?

- A

 Duplicates at source
Would make the multiple untidy, and spot checks confirm the individual rows are correct. A duplicate makes a row wrong, not a total.
- B

 Subtotals counted too
902,431,118 is exactly 3 x 300,810,373. Three grouping sets, three counts of every sale. Filter on the level flag.
- C

 The load ran three times
Would triple the row count as well, and the individual rows would then be duplicated rather than correct.
- D

 A fan-out
Would also triple, and again the rows themselves would be wrong. Every row here is right and there are too many of them.

THE METHOD

Question 2 answers this alone. An exact factor of three, with correct individual rows, is the signature of a result being counted once per level.

A deduplicated customer table is 1,847 rows short of the distinct customer count in the source. The deduplication keeps one row per customer, ordered by last_updated descending.

The missing customers all have several source rows sharing the same last_updated value. No errors are raised, and rerunning the job produces a different set of survivors.

What is the cause?

- ☐ A The source contains 1,847 customers with a null customer key, which the deduplication is dropping.
- ☐ B The deduplication is filtering rather than ranking, so customers failing the filter are removed entirely.
- ☐ C last_updated is not unique within a customer, so the ordering is incomplete and the survivor is arbitrary.
- ☐ D The partition key is wrong, so rows from several customers are competing for one surviving slot.

◆ PAUSE AND DECIDE

What is the cause?

A Null keys

Plausible and it would not explain a different result on each run. Nulls are dropped consistently, not arbitrarily.

C A tie in the ordering

The ties are stated, and a tie makes the survivor arbitrary. But an arbitrary survivor is still a survivor. this loses whole customers.

B Filtering, not ranking

A real defect, and it would lose the same customers every time. The changing survivors point elsewhere.

D The wrong partition key

Several customers share one slot, so all but one of each group is deleted. That is why whole customers vanish.

THE METHOD

Question 1 separates C from D. A tie changes WHICH row survives; it never changes HOW MANY. Losing whole customers means they were competing with each other.

A margin report is built by joining every sales line to the price that applied at the time. It returns 4,975,095 rows against 7,069,736 sales lines, and reconciles internally to the penny. The missing lines are concentrated in older products and in the first weeks of the period. Every line that is present carries a correct price.

What is the cause?

- ☐ A An inner join, and 2.09 million sales predate their product's first recorded price.
- ☐ B A date cast is dropping a time component, so afternoon sales fall outside their interval.
- ☐ C The price intervals overlap, so a deduplication downstream removed the extra matches.
- ☐ D Products delisted before the period began have no price rows at all.

◆ PAUSE AND DECIDE

What is the cause?

- A

 Inner join, no price yet
Price history begins partway through. Everything sold before a product's first price matches no interval and is dropped.
- B

 A date cast
Would lose rows uniformly across the period. These losses cluster at the start, which points at history rather than time of day.
- C

 Overlapping intervals
Produces too many rows, not too few. Overlap fans out; it is the opposite symptom.
- D

 Delisted products
Close, and it predicts losses spread across the period wherever a delisted product sold. The clustering at the start is the difference.

THE METHOD

Question 4 does the work. A loss that clusters at the beginning of a period is almost always a history that starts later than the facts do.

A view-to-basket conversion rate computed in the warehouse reads 31.7%. The app's own client-side analytics, which tracks taps in sequence, reads 17.9%. Both are reproducible over the same period and the same events.

The warehouse figure is sessions containing both events, divided by sessions containing a view.

Why do they disagree?

- ☐ A The session rule is merging two visits, so unrelated events appear inside one session.
- ☐ B Duplicate events in the warehouse feed are inflating the numerator.
- ☐ C The warehouse figure tests whether both events occurred, not whether one followed the other.
- ☐ D The two systems categorise event types differently, so 'view' means different things.

◆ PAUSE AND DECIDE

Why do they disagree?

A Sessions merging

Worth checking, and the session count is flat from 15 minutes to four hours on this data, so 30 is nowhere near a boundary.

C Presence, not sequence

38,125 sessions contain both events; only 21,574 have the add after the view. That ratio is the discrepancy exactly.

B Duplicate events

Would inflate both figures, and the client-side count is the lower one. Duplicates also fail a grain check, which this passes.

D Different categorisation

A real hazard across two systems, and it would not produce a one-directional gap of this precise size.

THE METHOD

This is a values question, not a rows question. Both figures are computed over the same sessions; they disagree about what the word 'then' means.

A category hierarchy report resolves 4,870 of the 4,912 categories in the dimension. The 42 missing ones are live categories with sales against them. The query completes in under a second and raises no error.

The walk starts from the root categories and descends. A depth cap of 20 is set and nothing reaches it.

What is the cause?

- ☐ A The depth cap is truncating the walk before it reaches the deepest branches.
- ☐ B The 42 categories have null parent keys, so they are neither roots nor children.
- ☐ C A filter on the recursive term is excluding them after the first pass.
- ☐ D They sit beneath a cycle, so their parents are inside a loop the downward walk can never enter.

◆ PAUSE AND DECIDE

What is the cause?

A The cap truncated it

Ruled out by the scenario: nothing reaches the cap. And a truncated walk would lose whole deep branches, not a scattered 42.

C A filter on the recursive term

A real defect that loses subtrees. It would normally lose them consistently by some attribute, and nothing here suggests one.

B Null parents

A null parent usually makes a node a root, so it would be included rather than excluded. It is worth checking and is not this.

D Beneath a cycle

A node in a cycle has its parent inside the cycle, so a walk from the roots can never arrive. It finishes fast and omits silently.

THE METHOD

The tell is 'completes in under a second and raises no error'. A downward walk meeting a cycle does not hang; it terminates early and omits. Only an upward walk loops.

A running total of basket value within each transaction is flat across several consecutive rows, then jumps. It happens on roughly 31% of transactions and never on the others.

The affected transactions all have two or more lines recorded within the same second. The window is ordered by the sale timestamp.

What is the cause?

- ☐ A The timestamps are stored in different time zones, so ordering is inconsistent within a transaction.
- ☐ B The partition key is missing, so the running total is accumulating across transactions.
- ☐ C The frame is RANGE rather than ROWS, so all rows with an equal ordering value share one frame.
- ☐ D The measure is null on the affected rows, so SUM skips them and the total does not move.

◆ PAUSE AND DECIDE

What is the cause?

A Time zones

Would reorder rows rather than flatten the total, and it would not correlate with ties inside one second.

C RANGE, not ROWS

A RANGE frame includes every peer with an equal ordering value, so tied rows all show the total after the last of them.

B No partition key

Would make the total climb across every transaction and never reset. The symptom is a plateau within a transaction, not across.

D Null measures

Would make the total flat on exactly the null rows. The scenario ties it to timestamp collisions, which is a different population.

THE METHOD

The correlation is the whole diagnosis: the defect appears exactly where the ordering value ties. That is a frame problem, and nothing else behaves that way.

A rebuild is compared against the original using a full outer join across all 34 columns, counting rows where any column differs. It reports 118,000 differences out of 7 million.

Spot checks show the flagged rows are identical in every visible respect. All of them contain at least one null.

What is the cause?

- ☐ A The rebuild genuinely differs on those rows, and the difference is in a column not shown by the spot check.
- ☐ B A join compares values, and NULL = NULL is unknown, so identical rows containing a null are flagged.
- ☐ C The two tables have their columns in a different order, so the comparison is misaligned.
- ☐ D A numeric precision difference is making values unequal below the displayed decimal places.

◆ PAUSE AND DECIDE

What is the cause?

A A genuine difference

Would not correlate perfectly with the presence of a null. That correlation is the diagnosis and it points away from real drift.

C Column order

Would flag nearly every row rather than 1.7% of them, and the flagged rows would differ visibly.

B NULL never equals NULL

In a join. Use EXCEPT both ways, where whole-row comparison treats nulls as equal and needs no per-column handling.

D Precision

A real hazard with floats, and it would correlate with numeric columns rather than with the presence of a null.

THE METHOD

'All of them contain at least one null' is doing all the work. A defect that correlates exactly with nullness is a three-valued-logic defect, every time.

A regional model has twelve assertions, all passing on every build for eight months. They check row counts, distinct keys, non-negative measures, and that each layer's total equals the next layer's.

Finance find the model is 3% below the group figure and has been since March. Every assertion still passes. Nothing in the model has changed.

What is the suite missing?

- A Every assertion compares the model against itself. None compares it against a source outside the model.
- B The assertions run on a fixture rather than on production data, so they never see the real figures.
- C The tolerance on the total comparisons is loose enough that a 3% gap passes as equal.
- D Twelve assertions is too few for a model of this size, and the coverage is simply thin.

◆ PAUSE AND DECIDE

What is the suite missing?

- A

 Nothing reaches outside
Each layer agreeing with the next proves the pipeline is consistent. It cannot detect something excluded before the first layer.
- B

 A fixture, not production
The figures are production figures. The problem is what the assertions compare against, not what they read.
- C

 Loose tolerance
A 3% gap fails any sane tolerance. The internal totals agree exactly, because the missing rows are absent from all of them.
- D

 Too few assertions
Twelve of one kind is one kind. A thirteenth internal check passes for the same reason the other twelve do.

THE METHOD

Question 3, and the answer is the least useful kind of yes. A chain of comparisons with no anchor agrees perfectly while describing the wrong population.

The eight, and the question that cracked each

SYMPTOM	QUESTION
Exactly 3x the correct total	2. An exact integer multiple means per-level counting.
Whole customers missing after a dedup	1. A tie changes which row survives, never how many.
2.09m rows dropped, clustered at the start	4. Loss at the beginning means history starting later.
Two systems, 77% apart, same events	1. A values problem: they disagree about 'then'.
42 nodes missing, fast, no error	4. Fast completion rules out a loop and implies omission.
A running total plateaus on 31% of rows	4. It correlates with ties, which is a frame problem.
118,000 identical rows flagged as different	4. It correlates with nulls, which is three-valued logic.
12 green assertions, 3% below finance	3. Reconciles with itself, so the population is wrong.

THE TAKEAWAY

Four questions cracked eight defects drawn from seven different modules. Not one of them required knowing which module it came from.

What the eight have in common

- ◆ **None of them raised an error.** All eight ran, returned rows, and produced a plausible number.
- ◆ **Six of the eight reconciled against themselves,** which is why they survived review and publication.
- ◆ **And every one was diagnosable from the symptom,** before anybody opened the SQL.
- ◆ **Which is the skill.** The techniques are lookupable; the mapping from symptom to cause is not.

THE TAKEAWAY

The observation that cracked most of these was a correlation: with ties, with nulls, with the start of a period. Correlations are visible without reading any code.

What does the defect correlate with? Ties, nulls, edges, or the start of a period. Each names a cause.

Four correlations, and what each names

THE DEFECT APPEARS WHERE...

LOOK AT

...the ordering value ties

The frame (RANGE vs ROWS), or an incomplete ORDER BY.

...a value is null

Three-valued logic. NOT IN, a join comparison, AVG's denominator.

...a group begins or ends

A missing PARTITION BY, or an edge treated as an exception.

...the period begins

A history that starts later than the facts, and an inner join.

...a container is empty

An expansion with an implicit inner join deleting the parent.

THE TAKEAWAY

Five correlations, and finding which one applies costs one GROUP BY over the affected rows. It is the cheapest diagnostic step there is.

Bisecting a query

```
1 WITH a AS ( ... ),      -- 7,069,736 as expected
2     b AS ( ... ),      -- 7,069,736 as expected
3     c AS ( ... ),      -- 4,975,095 <- it happened here
4     d AS ( ... )       -- 4,975,095
5 SELECT * FROM d;
6
7 -- SELECT COUNT(*) FROM each block in turn.
8 -- The first block where the count is wrong is the defect.
```

3 Module ten's debugging property. A stacked CTE lets you count at every level without restructuring anything.

THE TAKEAWAY

Bisection turns 'somewhere in 200 lines' into 'in this block'. It needs the query to be built as named blocks, which is why module ten argued for that shape.

When nothing correlates

- ◆ **The affected rows have nothing in common**, which usually means you have not computed the right property yet.
- ◆ **Try the ones people forget.** Position within the group, group size, and whether the row is first or last.
- ◆ **And try the negation**, because a defect affecting everything except one class is the same finding inverted.
- ◆ **If it genuinely correlates with nothing, suspect ordering**, because a non-deterministic result has no stable affected set at all.

THE TAKEAWAY

'Nothing correlates' and 'the affected set changes between runs' are the same observation. Run it twice before concluding that the defect is structureless.

The properties worth computing

```
1 -- the usual four
2 is null, ties in the ordering, first in group, date position
3
4 -- the ones people forget
5 COUNT(*) OVER (PARTITION BY k)      AS group_size,
6 ROW_NUMBER() OVER (PARTITION BY k
7                     ORDER BY ts)     AS position_in_group,
8 ROW_NUMBER() OVER (PARTITION BY k
9                     ORDER BY ts DESC) AS position_from_end,
10 LENGTH(some_text_column)           AS length,
11 some_value = 0                     AS is_zero
```

- 5 Group size catches the single-row group, which is 8.4% of Kestrel's sessions and behaves unlike every other group.
- 11 And zero against null is a distinction module two spent a module on. Test both separately.

THE TAKEAWAY

Ten properties, one GROUP BY. The one that separates the affected rows from the rest is your diagnosis, and it is usually in this list.

Bisection, and when to use it

- ◆ **When the correlation is unclear**, or when the affected rows have nothing obvious in common.
- ◆ **Count at every level**, and also sum a measure, because a block can preserve the row count and change the values.
- ◆ **It needs named blocks**. A nested subquery has to be restructured before it can be bisected at all.
- ◆ **Which is a reason to write them that way**, before you need to, rather than during an incident.

THE TAKEAWAY

Counting alone misses a block that preserved the grain and changed a value. Count and sum at every level, and the two together localise almost anything.

Reproducing it small

```
1 -- take the smallest input that still shows the defect
2
3 one affected key           the defect reproduces
4   + one unaffected key      the contrast is visible
5   + the tied / null / empty row that correlates
6
7 -- usually six to ten rows.
8 -- now the expected answer is computable by hand.
```

- 7 Once the answer is computable by hand, you can be certain rather than persuaded, which is the point of reproducing small.

THE TAKEAWAY

A defect reproduced on eight rows is understood. A defect observed on seven million rows is described. The difference is whether you can compute the right answer yourself.

When you cannot reproduce it

- ◆ **First suspect non-determinism**, because a defect that will not reproduce is often a query with several permitted answers.
- ◆ **Run it twice and diff.** Module eighteen's EXCEPT both ways, and a difference settles it immediately.
- ◆ **Then suspect the data moved**, because the rows that were wrong yesterday may have been corrected overnight.
- ◆ **So snapshot the inputs when a defect is reported**, before anything else. It is the cheapest thing in this module.

THE TAKEAWAY

A defect that vanished is usually still there. Snapshotting the inputs at the moment of the report costs a CREATE TABLE AS and saves the entire investigation.

Writing up a diagnosis

- ◆ **The symptom, with numbers.** 'Total was 3x' rather than 'the report was wrong'.
- ◆ **The correlation.** 'Affected rows were exactly those with a tied sold_ts', which is the evidence.
- ◆ **The cause, in one sentence,** and the assertion that would have caught it, which is the durable part.
- ◆ **And that assertion goes into the suite,** or the same defect returns in a year with a different symptom.

THE TAKEAWAY

A write-up without the assertion is a story. With it, the class of defect is closed rather than this instance of it.

The confusions, settled

BELIEF	THE TRUTH
Read the SQL first	Read the row counts first. Two thirds is eliminated for free.
A wrong number means a wrong query	Six of the eight here were consistent with themselves.
A tie explains missing rows	A tie changes which row survives, never how many.
A cycle makes a query hang	Downward it omits silently. Only upward loops.
It reconciles, so it is right	It reconciles, so the arithmetic is right. The population may not be.
The defect fixed itself	It correlates with data, and the data moved. It will return.

THE TAKEAWAY

Rows three and four are the two counterintuitive ones, and both appeared in this module's diagnostics because both are reliably got wrong.

Five habits

- ◆ **Count before you read.** Rows in, rows out, total in, total out, and the ratio to six decimal places.
- ◆ **Ask what the defect correlates with,** which is one GROUP BY and names the cause more directly than the SQL will.
- ◆ **Snapshot the inputs when a defect is reported,** before investigating, because the data will move.
- ◆ **Reproduce on ten rows,** so that the right answer becomes computable by hand rather than argued about.
- ◆ **And leave an assertion behind,** because a diagnosis without one closes an instance rather than a class.

THE TAKEAWAY

Four of the five happen before the query is opened. The instinct to read the SQL first is the single most expensive habit in this module.

Diagnosing somebody else's query

- ◆ **Ask what they believe about the grain**, first, because half of all defects are a grain they never stated.
- ◆ **Ask what the number should be**, and where that figure comes from, because an anchor is often the missing piece.
- ◆ **Do not start by reading their SQL**, which is slow, and which anchors you to their mental model of it.
- ◆ **The four questions work on a query you have never seen**, which is most of their value.

THE TAKEAWAY

The four questions need the result and the source, not the query. You can diagnose a query written by somebody who has left, in a dialect you do not use.

A defect is a disagreement between what a query computes and what somebody believed it computed.

Finding the correlation

```
1 -- take the affected rows and the unaffected ones,
2 -- and compare them on everything you can think of
3
4 SELECT is_affected,
5        COUNT(*)                               AS n,
6        AVG(CASE WHEN some_key IS NULL THEN 1 ELSE 0 END) AS null_rate,
7        AVG(rows_sharing_ts)                     AS mean_tie_size,
8        MIN(cal_date), MAX(cal_date),
9        AVG(CASE WHEN rn = 1 THEN 1 ELSE 0 END)   AS first_in_group
10 FROM flagged
11 GROUP BY is_affected;
```

4 One GROUP BY, two rows of output, and the column where the two rows differ most is your diagnosis.

THE TAKEAWAY

The cheapest diagnostic there is. Flag the affected rows, group by the flag, and read across for the property that separates them.

Things that are not the cause

- ◆ **'The data is bad.'** Sometimes true, and it is the conclusion after diagnosis rather than a substitute for it.
- ◆ **'It must be a rounding issue.'** Rounding produces differences in the last decimal place, not 3% or 77%.
- ◆ **'The two systems just count differently.'** They do, and the question is how, which is the actual answer.
- ◆ **'It fixed itself.'** A defect that correlates with data will disappear when the data changes and return.

THE TAKEAWAY

All four are ways of closing an investigation without finishing it. The last is the expensive one, because the defect is still there and now has a history of being dismissed.

The method, in four steps

01

Classify

Too many rows, too few, or wrong values? One row count, and two thirds is eliminated.

02

Check the ratio

Exact small integer means per-level counting. Untidy means data.

03

Correlate

Flag the affected rows and group by the flag. The column that separates them is the answer.

04

Then read the SQL

By now you have two or three candidates, and you know what to look for.

THE TAKEAWAY

Step four is last on purpose. Reading the query first means reading all of it with no hypothesis, which is how an afternoon disappears.

You have diagnosed eight. **Module twenty-four** hands you twelve more, and does not tell you the answers.